

The Accumulator Pattern

CSCI 1101

2024-02-23

The Accumulator Pattern (review)

A very general pattern for computing with data in a list is the **accumulator pattern**.

We use an **accumulator** variable to keep track of the information that we want to learn about the list and use a *loop* to update this information for each list item.

The general structure of the accumulator pattern is:

```
acc = <initial_value>           # initialize accumulator
for item in <list> :             # iterate over list
    acc = <update_function>      # update accumulator
<do_something_with_acc>         # use accumulator value
```

The **initial value** of the accumulator should be whatever is the “right answer” if the list is empty.

The **update function** is an expression involving the current accumulator value and current list item that specifies the next accumulator value.

Activity: Deduplicate

To *deduplicate* a list we keep just one copy of each item.

Write a function `deduplicate : (list a) --> list a` that returns the list containing only the *first* copy of each item in the argument list.

For example:

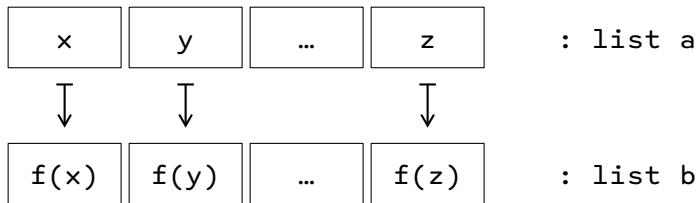
```
> deduplicate([1,2,1,3,2,4])  
[1, 2, 3, 4]
```

Remember to ask yourself:

- What is the deduplicated version of an empty list?
That is the accumulator *initial value*.
- If you know the deduplicated version of the list so far and the current item, then how do you extend the deduplicated list?
That is the accumulator *update function*.

List Mapping

A common programming task is to take a function $f : (a) \rightarrow b$ and apply it to each item of a list, collecting together the list of results. This **list mapping** operation has type $(\text{list } a) \rightarrow \text{list } b$.

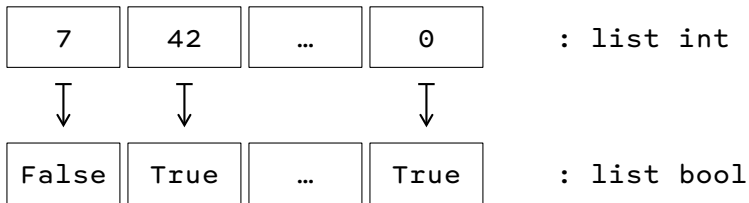


We can do *list mapping* using the *accumulator pattern*, where:

- the *initial value* is the empty list (why?),
- the *update function* appends to the accumulator list the result of applying the function f to the current item in the argument list.

Example: Mapping Even

Suppose we want to map the predicate of a number being *even* over a list, to make a function `map_even : (list int) --> list bool`.



Let's do this using the mapping version of the accumulator pattern.

```
def map_even(nums) :  
    results = []  
    for num in nums :  
        results.append(num % 2 == 0)  
    return results
```

Activity: Back-Words

Write a function `reverse_word : (str) --> str`
that reverses the order of a string:

```
reverse_word('Bowdoin')  
'niodwoB'
```

Now write `reverse_words : (str) --> str`
that takes a sentence and reverses each word in the sentence, but leaves the word order unchanged. You can assume that the words are separated by one space with no punctuation.

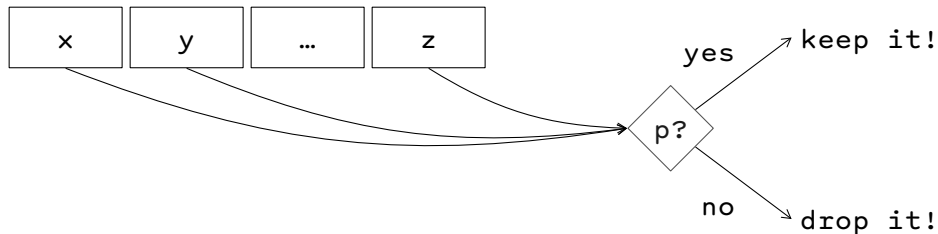
```
reverse_words('for whom the bell tolls')  
'rof mohw eht lleb sllot'
```

Hint: use list mapping.

List Filtering

A common programming task is to take a predicate $p : (a) \rightarrow \text{bool}$ and apply it to each item of a list, keeping only those that “pass the test”.

This **list filtering** operation has type $(\text{list } a) \rightarrow \text{list } a$.



We can do *list filtering* using the *accumulator pattern*, where:

- the *initial value* is the empty list (why?),
- the *update function* appends to the accumulator list the current item of the argument list *only if it passes the test* p .

Example: Filtering Even

Suppose we want to filter a list of numbers by the *even* predicate, to make a function `filter_even : (list int) --> list int`.

Let's try to do this using the filtering version of the accumulator pattern.

```
def filter_even(nums) :  
    evens = []  
    for num in nums :  
        if num % 2 == 0 :  
            evens.append(num)  
    return evens
```


Activity: Remove Vowels

Write a function `remove_vowels : (str) --> str` that takes a string and returns the string that results from removing all vowels:

For example:

```
> remove_vowels("Bowdoin!")  
'Bwdn!'  
> remove_vowels("It was the best of times")  
't ws th bst f tms'
```

Hint: use list filtering.

Nested List Iteration

In the accumulator pattern, the update function could itself be an instance of the accumulator pattern. This is known as **nested list iteration**.

The pattern is:

```
outer_acc = <outer_initial_value>
for item_1 in <list_1> :
    inner_acc = <inner_initial_value>
    for item_2 in <list_2> :
        inner_acc = <inner_update_function>
    outer_acc = <outer_update_function>
<do_something_with_outer_acc>
```

This is just one copy of the accumulator pattern sitting in the *update function* position *inside of* another copy of the accumulator pattern.

Example: Multiplication Table

Using *nested list iteration* we can calculate the “multiplication table” for two lists of numbers:

```
def products (nums_1 , nums_2) :  
    ''' sig: (list int , list int) --> list (list int) '''  
    outer_acc = []  
    for num_1 in nums_1 :  
        inner_acc = []  
        for num_2 in nums_2 :  
            inner_acc.append(num_1 * num_2)  
        outer_acc.append(inner_acc)  
    return outer_acc
```

Activity: Compound Words

Write a function

`compound_words : (list str , list str) --> list str`
that takes a list of *prefixes* and a list of *suffixes* and returns the list of all compound words resulting from concatenating a prefix with a suffix.

For example:

```
> compound_words(['play', 'slack'] , ['ing', '-off'])  
['playing', 'play-off', 'slacking', 'slack-off']
```

Hint: use nested list iteration.

To Do

Finish *Project 1 - The Turtle Module* on CodeRunner by midnight tonight.

Finish *Homework 5: Lists and Loops* on CodeRunner by 11:00AM next Thursday.